

PART 07

What runs now

The workforce changed between 26 and 30 August 2026 in five places that matter to anybody using it: what an agent is told, who serves the model call, what it may do about a particular subject, how often it may act, and what a person has to decide before it acts more. This part is that difference.

IN THIS PART

- 07.1 · The prompt, as it is assembled today
- 07.2 · Free models first, and a paid floor of three
- 07.3 · What an agent is told about this task
- 07.4 · Boundaries: what, and about whom
- 07.5 · Switching the workforce on
- 07.6 · Pace — how often, as against how much
- 07.7 · Autonomy as a decision with a record
- 07.8 · A question that was never answered
- 07.9 · The induction of a hired agent
- 07.10 · What all of it changes at a desk

The prompt, as it is assembled today

An agent's instruction is not one block of text. It is 9 labelled regions assembled per task, of which exactly one is a person's own writing; the other eight are read from live state, and editing a copy of any of them would either be overwritten on the next run or quietly diverge from what the rest of the roster is told.

The regions are what the Prompt screen shows and what `composePrompt()` in `services/agents/runner.ts` builds. Two of them are new since the earlier books, and one of the two is the reason the others were relabelled.

REGION	WHERE THE WORDS COME FROM	PRESENT	AUTHORED BY
01 Its instructions	The agent's own text, or the ten shipped sections run together in order.	Always	a person's own
02 What it is relied on for	The skills on this agent, edited on the same screen.	When it has any	live state
03 Who Dakyworld is	services/dakyworld.ts — the same for every agent.	Always	live state
04 The company's details	Settings → System. Change it there and every agent and document follows.	Always	live state
05 How it writes	services/dakyworld.ts — the brand voice.	Agents that write for outside	live state
06 What Dakyworld holds	Shared memory, recalled for this task's subjects. Every agent is shown these.	When anything was recalled	live state
07 What it already knows	Its own memory, recalled for this task's subjects.	When anything was recalled	live state
08 How it does the work	services/agents/runner.ts — the same four passes for every agent, seeded or hired.	Working runs only	live state
09 How it works here	Generated from live state — tool etiquette, dry run, and the size of the roster.	Always	live state

THE 10 LAYERS NOW CARRY HEADINGS

An agent nobody has rewritten is instructed by the 10 shipped sections, joined in order. Until 29 August they were joined as 10 anonymous paragraphs — a wall in which the rule about money, the definition of finished and the description of the craft all looked alike. They are now joined under headings (`LAYER_HEADINGS` in `services/agents/authored.ts`), at a cost of roughly thirty tokens:

THE HEADING THE LAYER IS JOINED UNDER	THE LAYER IT IS
01 Who you are	role
02 What you are for	mission

	THE HEADING THE LAYER IS JOINED UNDER	THE LAYER IT IS
03	What is yours and what is not	scope
04	What you may treat as fact	dataRules
05	How you use what you have been given	tools
06	The rules you work under	policy
07	How this job is done well	process
08	When to stop and ask	escalateWhen
09	What you produce	output
10	What to keep	memory

The words did not change — the layout did

This is presentation, not rewriting. An agent whose wording the Owner has replaced outright is still returned exactly as written, unheaded and untouched, because the whole point of that field is that a person's prose reaches the model as their prose.

METHOD — FOUR PASSES, ONE COPY, EVERY AGENT

The reason both changes were needed at all is 07.2: every agent turn now starts on a free model. A strong model reads a paragraph of craft doctrine and infers the procedure — look at the record before forming a view, put a source under a figure, read the draft back before handing it over. A weaker one does the thing the paragraph talks about and skips the procedure nobody wrote down, and the failure reads as carelessness rather than as a missing instruction: an agent that answered from the brief without opening the record, a number with nothing under it, a letter nobody checked.

1 Establish

Get what is actually on the record in front of you before forming a view — the lead, the client, the project, the thread. A brief is somebody's summary of a record, not the record. Where a fact cannot be got, write down that it could not, rather than working around the hole quietly.

2 Decide

Say what was concluded and name the one or two facts that decided it. A conclusion with no fact under it is a preference, and the next person to read it has to derive it again from scratch.

3 Produce

One finished thing, of the kind named under *What you produce*. Finished means somebody could use it as it stands — not an outline of it, not a description of what it would contain.

4 Verify

Read the work back against the record: every figure to its source, every name as the record spells it, every claim to the thing that supports it. Then say, in one line, which part of it you are least sure of — that line is worth more to the reader than another paragraph of the work itself.

Working runs only. A colleague answering a consult is answering one question, not producing a finished thing, and does not get these four. And METHOD deliberately says nothing about tools, escalation, memory or who to ask: all four already have paragraphs in the working region, and a prompt that says the same thing twice in two sets of words is how a model averages two instructions into neither. It is a region rather than a seed field precisely so that it also reaches the agents the Agent Creator hires.

Every seed's process is a numbered workflow now. 48 of the 51 were rewritten by hand on 29 August: the first step names what to read, the judgement that was already there is kept sentence for sentence, and what was added is the order — and the step that was missing. outreach.writer and outreach.followup are deliberately untouched: their process *is* the cold email and follow-up doctrine, which is prose for a reason.

How a prompt change reaches a live database

ensureAgents() only ever creates, so re-wording a seed changes nothing on a database that already has that agent. refreshUneditedSeedPrompts() runs on every boot and carries a changed seed onto any agent whose wording the Owner has **not** rewritten — 48 of them on the first boot after the rewrite, in production. That is the mechanism for any future prompt change; a second marked one-off pass is not needed and should not be added.

07.2

Free models first, and a paid floor of three

OpenRouter publishes models that cost nothing per token. They are real models and they are also the least reliable thing available: a free endpoint is shared, so it queues, rate-limits, and some of the time simply does not answer. One of them as *the* model is a system that stops working at busy times. Three in a row with a paid floor behind them is a system that costs nothing most days and never stops.

Rungs 1–3 • free openrouter.freeModels · freeLadder()	Three free, tool-capable models taken from the account's own catalogue, tried in order. One attempt each and a short clock — 60 seconds for one-shot work, 120 for an agent turn. Waiting out a queue is right when the vendor is the only one who can do this and wrong when the next free model is one line down.
Floor 1 • Claude Anthropic SDK	The first paid vendor, and the one that keeps the patient behaviour: four attempts and ninety seconds of backoff.
Floor 2 • ChatGPT OpenAI chat completions	A floor of one under a ladder built entirely out of endpoints that fail is a single point of failure in the place least able to afford one: a run that survived three busy free models and then met a rate-limited Claude died with two connected vendors sitting unasked.
Floor 3 • Gemini :generateContent	The agent loop speaks all three wires and translates each into its own Anthropic-shaped state, so a conversation can start on a free rung and finish on Gemini with its checkpoint intact.

01 It is on by default

It shipped as an opt-in on 28 August and was switched on by default the next day. An opt-in nobody has opted into is a feature that does nothing: a deployment that never opened the Settings screen was paying for every agent turn while three free models sat there.

02 Unset, empty and unreadable are three different states

Unset is the shipped ladder. A stored empty list is free models deliberately off, and must survive a deploy. An unreadable row falls back to the shipped ladder rather than quietly starting to pay for things. Conflating any two of them is a money bug.

03 The ladder is refreshed from the account, not from a source file

A free slug written down in a source file is the most perishable id OpenRouter publishes. The shipped seed is replaced at the first boot with a key by the three best free tool-capable models the account's own catalogue lists — and a model without tool support writes perfectly well and fails every agent task.

04 The ladder replaces OpenRouter's model — it does not precede it

Slipping a paid call in between the free rungs and the floor would be a bill nobody asked for at exactly the moment free capacity was short. Switched off, nothing else about the model layer changes.

05 A key-level refusal does not climb

401, 402 and 403 are true of every model on the account, so climbing is three calls into the same wall and a slower failure. 400, 404, 429, 5xx and silence are true of that model and say nothing about the next one.

06 A rung is priced at zero, from the ladder rather than from the id

Most free ids end in : free and not all do, and a naming convention is not a price. Left unpriced a rung would fall through to the floor rate, which is deliberately the dearest known — so a free day would read as the most expensive one this company has ever had and trip every ceiling on money nobody spent.

07 A turn that says nothing is not a finished turn

A model that returns neither text nor a tool call is what a busy or degraded endpoint does instead of a clean error, and the loop used to accept it as finished because nothing threw — a task reporting success having done, and said, nothing at all. It is now treated exactly like a failed rung: climb the ladder, hand to the floor, and give up only when nobody is left to ask, keeping the conversation so a re-run continues rather than starts over.

A refusal is still never routed around

Everything above is about a vendor that *could not* answer. A model that *declines* is answering, and asking the next one until somebody agrees is how a policy gets laundered. That distinction predates the ladder and survives it.

Two defects this uncovered, both invisible and both live

The model name in the agent loop was resolved once from whichever vendor came first, so **every handover to Claude was asking Anthropic for OpenRouter's model id** — the failover that exists to save a run would have died on the model name. It survived testing because the fake Anthropic echoes a Claude id whatever it is asked for, so “Claude finished the run” passed while the request said otherwise; the check now reads the model out of the request body. And the vendor address was captured at import in one half of the model layer and read per call in the other, so a harness repointing a vendor got a frozen address in one half and a live one in the other — a check that passes while testing nothing, and on a machine with a real key one that spends money. There is one function for it now, imported by both halves: two correct copies of the same fact are one copy away from two different facts.

07.3

What an agent is told about this task

Two things were added to the top of a run, both cheap, both aimed at the same waste: a model working out from scratch, on every task, what it could simply have been told.

WHICH OF ITS TOOLS THIS BRIEF IS PROBABLY ABOUT

An agent is handed everything its toolkit grants, with a catalogue description each — eighteen of them for the Cold Lead Writer — and no indication of which the brief calls for. The three whose purpose most overlaps the task's own words are now named in a sentence on the brief.

LIKELY Scored, never filtered

RUNNER.TS · LIKELYTOOLS()

WHAT IT DOES	Ranks the granted tools against the words of the title and brief, keeps the best three, and writes them onto the brief under the line: <i>this is a guess from the wording and nothing more — your whole toolkit is available, and if the right tool is not in this list, use it anyway.</i>
WHAT IT NEVER DOES	Remove or reorder anything. A tool the model cannot see is a tool it cannot use, so a bad ranking would be a silent loss of capability rather than a worse suggestion.
ZERO OVERLAP	Says nothing at all. Three tools the task never mentioned, presented as the likely ones, is a wrong answer where there was previously an honest absence.
ON THE BRIEF	Deliberately not in the system prompt, which carries a cache breakpoint and is identical for every task this agent runs. Per-task text there would write a new cache entry each time and read none — the expensive half of caching with none of the saving.

HOW MANY COLLEAGUES ONE TASK MAY ASK

Three consults is a hard question being worked properly; ten is an agent canvassing the building instead of deciding, at a model call each. One number for every task was wrong in one direction only — too few for the rare task where being wrong is expensive, which is exactly the task somebody marked urgent. The ceiling now varies by the priority the task already carries, and needs no new field and no new vocabulary to do it:

TASK	PRIORITY	COLLEAGUES	WHY
1	Urgent	5	Getting this wrong is expensive, and it is the task somebody marked urgent for exactly that reason.
2	Normal	3	A hard question being worked properly. The shipped default for everything unmarked.
3	Low	2	Ask once, decide, and get on with it.
Anything else		3	A priority not in the table falls back to the ordinary ceiling rather than to none.

It is overridable as a setting, where zero is a real limit rather than an unset value: no consults at this priority is something somebody may deliberately want, and the usual guard would hand them the default back without saying so. An unreadable override is treated as no override — a hand-edited setting must not stop the workforce consulting. The count itself stays one number on the checkpoint: a task has one priority, so only the ceiling varies.

DOWN THE CHART BEFORE SIDEWAYS ACROSS IT

An agent may delegate to its own reports as often as the work needs, and gets two hand0ffs across the chart on a whole task. When the roster search finds a match sitting under one of the agent's own reports, it now says so — and says to delegate to that report and let them route it. A search that named only the craftsman left an executive with two hand-offs and no route, which is how a whole-floor run spent both on specialists and never asked the three directors who own those lanes.

07.4

Boundaries: what, and about whom

Every agent carries a list of work it is explicitly not responsible for. Until August 2026 that list was correct, shipped, documented and unreachable — nothing ever wrote the column, so the

gate read an empty array on every agent on every database and the enforcement had never once fired.

A What is being done		B Who it is being done about	
WRITTEN AS	Tool-key globs, where * is the only wildcard and matches any run of characters, including none.	WRITTEN AS	Subjects — lead, client, project, proposal, invoice — matched against the records the task hangs off.
WHY A GLOB	The first version compiled each pattern as a regex, which is wrong twice: the unescaped dot in <code>Lead.update</code> also matches any character, and a pattern carrying a bracket — which the Agent Creator can write, since a hired agent's boundaries come from an approved design rather than from a seed — throws out of the permission gate that every tool call in the system passes through.	WHY IT IS SEPARATE	A tool key cannot say this. <code>email.draft</code> is the right tool for a stranger and for a client of two years, and what separates them is the record behind the task. A task carrying both a lead and a client is a crossing rather than an excuse: a lead that is also a client is a client, and the point of the boundary is that the Cold Lead Writer must not open a first message to one.
		COST	Read only when an agent actually declares one, so every other agent pays nothing for the feature. A call with no task behind it is never refused — there is no subject to be wrong about.

THREE CROSSINGS IN A ROW, AND THE WORD "ROW" IS LOAD-BEARING

A crossing is refused, counted and named. On the third **consecutive** one the agent is paused, and the message says which pattern was crossed and on which tool — because a paused agent whose card says nothing about why is the “fixable setting rendering as *Something went wrong*” failure this system has a standing rule against, and the fix is more often to correct the boundary than to scold the agent. The count is cleared by the agent's next allowed call, and by setting it back to Active, which *is* the human review those strikes were counting up to. Without that reset it was a lifetime tally: an agent crossing once a quarter was silently paused on its third occasion, months apart, under a message claiming three consecutive violations. A count that never resets is not a strike system, it is a slow fuse.

Surplus toolkits are now reported on every boot

When an agent was narrowed to one deliverable, the tools its old scope needed were left on its record on purpose — silently revoking a grant is invisible until the day something cannot be done, so the untick is a person's. The report naming them, though, was computed inside a marked one-off pass, so it ran exactly once, on the boot that carried the split, and never again. It is a plain read now, called at boot and returned with the roster so the Agents screen can show it. Ten narrowed agents are currently holding a tool their one job no longer needs, and until 30 August nothing anywhere had said so.

07.5

Switching the workforce on

Every specialist ships as a draft, and a draft picks nothing up. On a roster of 51 that made the ordinary first act — start the ones I want working — 51 visits to a drawer.

BULK Status, and only status

PATCH /API/AGENTS/BULK

DOES	Sets Draft, Active, Paused or Retired on up to 200 agents at once, clears their boundary strikes when the new status is Active, and answers with how many queued tasks can now actually start — which is the question somebody activating a batch is really asking.
NEVER DOES	Autonomy or dry run. Those decide whether software may act on a client without being asked; they are decided one agent at a time, with a reason, behind the guard in 07.7. A bulk control over them would be a single click that widened the whole workforce — exactly the thing this system is built not to have.
MEANS	Activating an agent only lets it pick up the queue it already has, at whatever autonomy it already holds — which, for everything on the roster as shipped, is level 1 with dry run on.

A REHEARSAL WAKES WHAT IT NEEDS AND PUTS IT BACK

The same problem met anybody opening the Rehearsal screen: greyed-out workflows and an instruction to switch eleven agents on by hand, which is not a test of the workforce but an errand standing in front of one — and which leaves behind a floor of agents switched on because of a test, quietly taking real work.

1 A draft is woken; a paused agent is not

A draft has simply never been switched on. Pausing is something a person *did* — it is precisely how the Owner stops an agent's standing work. Retired is the same, more so. Both are refused with the reason said out loud.

2 Waking is three columns, not one

Status, autonomy level and dry run. An agent set Active on a card that still says level 1 with dry run on is an agent that turns up, is handed the work, and is refused every tool it owns.

3 Only what it woke is put back

An agent that was already active keeps the level and the dry-run flag the Owner gave it. A run may therefore still contain an agent that prepared everything and carried out nothing — a true fact about the setup rather than a fault, which is why every prepared call now carries the sentence saying which gate held it.

4 What was woken is written down before anything is woken

On the rehearsal's own row. A process dying mid-rehearsal would otherwise leave the floor switched on with nothing anywhere recording that it should not be — the failure mode where a test quietly changes how the business runs. A boot-time sweep restores anything orphaned.

07.6

Pace — how often, as against how much

The spending ceilings answer *how much money* and say nothing about *how often*. Those come apart: an agent doing cheap work — reading records, filing tasks, drafting short things — can run all day well inside its budget and still act far more than anybody meant it to, and the only signal is a timeline nobody is watching.

CEILINGS	Per day, per week and per month, set on the agent. A pace ceiling is the one somebody sets when they mean “try this, but not fifty times”.
PERIODS	Calendar periods in UTC, matching the spending ceilings exactly, and for the reason written there: a ceiling that rolls over at a different instant from the one the reports total against is a ceiling the reports cannot explain. A week starts Monday — the same ISO week the escalation digest is keyed on.
WHAT COUNTS	A task whose start time falls inside the period. That time is rewritten on every claim, including a resume, so a task begun yesterday and continued today counts once, today. Deliberately kinder than counting pickups: a deploy interrupting a run five times is infrastructure, not the agent doing five things, and a quota that punished it would empty itself on a bad afternoon.
WHERE IT STOPS	In the runner's due-task pass, beside the budget check — the one place where stopping is free, because nothing has started and nothing is half-done.
WHAT IT LOOKS LIKE	The task stays QUEUED with its place kept, and begins on the tick after the period rolls over. Being at a ceiling is the guardrail working: it is not an error, and it does not change the task's status.

07.7

Autonomy as a decision with a record

The question an autonomy decision needs answered is *has this one earned it*, and before 30 August there was nowhere to look: the roster showed a level and a dry-run tick and nothing behind either.

HISTORY Every change, kept

Who changed the level or the dry-run flag, from what to what, when, and why. Raising what software may do to a client without being asked is a decision, and a decision with no record is one nobody can review, defend or reverse.

EVIDENCE Summed, never counted

Tasks by end state, tool calls, how many of them were previews, refusals, and how often this agent stopped to ask — all summed from ledgers that already exist. Accounting is explicitly allowed to fail silently so that it never breaks the work it accounts for, which means a stored counter would drift the first time a write failed. A promotion argued from a counter that has quietly lost count is the worst version of this feature.

Stopping to ask is counted off the transition into **BLOCKED** rather than off a task's current status, or a task that was blocked, answered and finished would read as never having stopped to ask at all. And an agent nobody has given work to reads as *no evidence* rather than as a row of zeroes: a window with none of something is not a zero, and an agent with no history is a very different thing from one that fails everything. Nothing on that screen promotes anything — the change itself goes through the ordinary route, gated on the same permission as before, and now leaves a history entry.

The queue is read before the level is raised

An agent at level 1 with dry run on has been *preparing* work all along — that is what dry run is for — and every prepared action is a decision somebody has not made yet. Raising the level does not carry those out, but it does mean the next one like them happens without being asked. Deciding that while a queue of exactly that thing sits unread is the shape of the mistake this guards against, so the prepared actions the agent is holding, oldest first and with the hours each has waited, are put in front of whoever is changing the level. A separate sweep reports prepared actions nobody has looked at for two days, by agent. A rehearsal's prepared actions are excluded from all of it: they are specimens rather than proposals.

07.8

A question that was never answered

An escalation is posted to Slack the moment it happens. That card is the right thing for the first hour and the wrong thing for the second week: it scrolls away, and a question nobody ever

answered was indistinguishable from one that was, because nothing anywhere counted them — while stopping to ask is the most important thing an agent ever does.

The agent stops escalate	The task goes to <code>BLOCKED</code> , keeps its checkpoint, and the question is marked as waiting on a person rather than merely existing in a status.
A card, immediately agents/escalationCards.ts	Posted to Slack and answerable there. Answering appends to the conversation the agent will rejoin as well as to the brief on the record, or it resumes at the moment it asked and asks again.
A digest, weekly agents/escalationDigest.ts	Every question still waiting, oldest first. A question older than seven days has stopped being a question and become a problem, and is named as one.
And a screen, always GET /api/agents/escalations	The same rows, without Slack. A workspace nobody connected, a secret nobody pasted, an app somebody removed — and the symptom would be questions nobody can see. Slack is never the only road.

Rehearsal questions are excluded from both the digest and the screen, for the same reason a rehearsal posts no cards: it is a test, and a test's questions are read on the rehearsal screen beside the run that raised them.

07.9

The induction of a hired agent

A hire arrives with a prompt, a toolkit and no memory at all, so its first task is worked with less context than the same agent will have on its second — and the two facts that would help most are known at exactly that moment and nowhere afterwards.

FACT Why you were employed The craft an agent actually said was missing, in its own words, and which agents said it. "That is the work you exist for; anything else is somebody else's and goes back with handOff."	LESSON Who already does some of this The closest crafts on the roster, with a percentage each — the same calculation the approver read on the hire card, so the new agent's picture of the roster and the person's are the same picture. Where a job sits between them, ask before deciding it is yours.
---	--

Both are facts from the hiring decision rather than lessons anybody invented: an induction that summarised other agents' finished work would be this system telling a new colleague what it guessed their job was like, which is worse than silence. Both are filed as the agent's **own** memory against *self* — recalled on every task for this agent and nobody else. Filed as shared, they would put one agent's induction into all 51 prompts for ever. And the write never throws: an agent that exists without its induction is a small loss, while a hire that half-happened because a memory write failed is a real one.

And the task that was waiting is told

The point of the whole loop. A task that stopped because nobody could do this is requeued with the news on its brief and in the conversation it will rejoin — the new agent named, with its key and its mission, and instructions to hand the work over or ask them a question and carry on with its own part. Left out, a filled gap is a new agent nobody thinks to use and a blocked task that stays blocked.

07.10

What all of it changes at a desk

The same mechanisms, read as symptoms a person actually meets.

WHAT YOU SEE

WHAT IT IS, AND WHAT TO DO

A task sits QUEUED and nothing is wrong	A pace ceiling, a spending ceiling, or an agent still in Draft. All three keep the task's place; the first two clear themselves when the period rolls over.
An agent is Paused and nobody paused it	Three boundary crossings in a row. The message names the pattern and the last tool. Either the boundary is wrong or the work is being sent to the wrong agent — setting it back to Active clears the count, and is the review.
"Pending a person's approval" on work you never saw	The gate held an outward or spending call and the task finished at <code>NEEDS_APPROVAL</code> . Every prepared call now carries the sentence explaining which gate held it, on the timeline and on the rehearsal screen.
A run started free and finished on Gemini	Working as designed: three free rungs, then Claude, ChatGPT, Gemini. The checkpoint carries across all of them, so the run continues rather than starting again.
A task "succeeded" having said nothing	Fixed. An empty turn is treated as a failed rung: climb, hand to the floor, and fail honestly if nobody is left — keeping the conversation so a re-run continues.
An agent holding a tool it no longer needs	Reported at every boot and with the roster. Unticking it is a person's call, deliberately: silently revoking a grant is invisible until the day something cannot be done.
A question in Slack nobody answered	It is on the weekly digest and on the Agents screen from the moment it was asked, with its age in days. Answering it resumes the task from where it stopped.
You want more of the workforce working	Activate in bulk; the reply says how many queued tasks can now start. Autonomy stays one agent at a time, with the waiting-approvals queue in front of you when you change it.